

Cómo funciona HTTPS: de la URL al candado

Qué pasa entre escribir una dirección y ver el candado, y qué garantiza ese candado

Resumen para estudiar. Cada dato lleva en gris su fuente; las siglas remiten a las referencias del final.

PUERTO POR DEFECTO 443 si la URL https no indica otro RFC 9110, §4.2.2	HANDSHAKE DE TLS 1.3 1 RTT TLS 1.2 necesitaba 2 RFC 8446 · RFC 5246	HASTA LA RESPUESTA 3 RTT TCP, TLS 1.3 y HTTP, sin el DNS (cuenta propia)	CONTROLES POR CERTIFICADO 4 firma, vigencia, revocación y emisor RFC 5280, §6.1.3
---	--	---	--

En una página

Del nombre a la conexión. El navegador traduce el nombre a una dirección IP con DNS, abre una conexión TCP al puerto 443 y sobre ella negocia TLS RFC 9110, §4.3.3. TCP gasta un viaje de ida y vuelta (RTT) antes de que el cliente pueda enviar datos RFC 9293, §3.5.

El handshake. En TLS 1.3, cliente y servidor acuerdan las claves en un solo RTT, y todo lo que sigue al ServerHello viaja cifrado RFC 8446, §1.2 y §2. En una conexión nueva, la primera respuesta HTTP llega a los 3 RTT.

La identidad. El certificado X.509 ata el nombre del servidor a una clave pública, y el servidor firma el handshake con la clave privada. El navegador sigue la cadena de firmas hasta una raíz que ya tiene RFC 5280, §6.1.

El alcance. HTTPS cifra la ruta, los encabezados y el cuerpo, y detecta a quien se haga pasar por el servidor. No oculta la IP, el nombre del sitio ni el tamaño y el ritmo del tráfico RFC 8446, §1.2 y §E.3 · RFC 9110, §4.3.3.

Lo que el candado no dice. Que el sitio sea honesto. Para emitir un certificado, Let's Encrypt solo le pide a quien lo solicita que pruebe que controla el dominio LE-validación.



El camino prepara el handshake; handshake y certificado determinan lo que HTTPS protege. Cada caja lleva a su sección.

01 La URL fija el destino; DNS y TCP abren el camino

El esquema https obliga a asegurar la conexión antes de enviar la petición: el servidor tiene que estar autenticado y la comunicación, cifrada y protegida contra cambios RFC 9110, §4.2.2.



Cada parte de la URL la usa una capa distinta. En azul, lo único que viaja cifrado, dentro de la petición HTTP MDN-TLS; el host viaja en claro (→ sección 04).

Paso 1: DNS. El resolutor traduce el nombre a una IP consultando servidores de nombres en el puerto 53 RFC 1035, §4.2. Puede necesitar varias consultas y un tiempo arbitrario, y guarda las respuestas en una caché RFC 1035, §2.2: por eso este resumen no lo cuenta entre los RTT. Una respuesta falsa puede llevar a otra IP; verificar el certificado detecta a quien controla la resolución RFC 9110, §4.3.4.

Paso 2: TCP. El cliente envía un SYN, el servidor responde con SYN,ACK y, al recibirlo, el cliente ya puede mandar datos junto con su ACK RFC 9293, §3.5, fig. 6. La conexión cuesta un RTT y prepara el camino para TLS.

Confusión típica. Creer que llegar a la IP y al puerto 443 prueba que se habla con el sitio. HTTPS no usa TCP ni el puerto para decidir la autoridad, porque quedan fuera de la comunicación asegurada; la prueba es el certificado RFC 9110, §4.3.3.

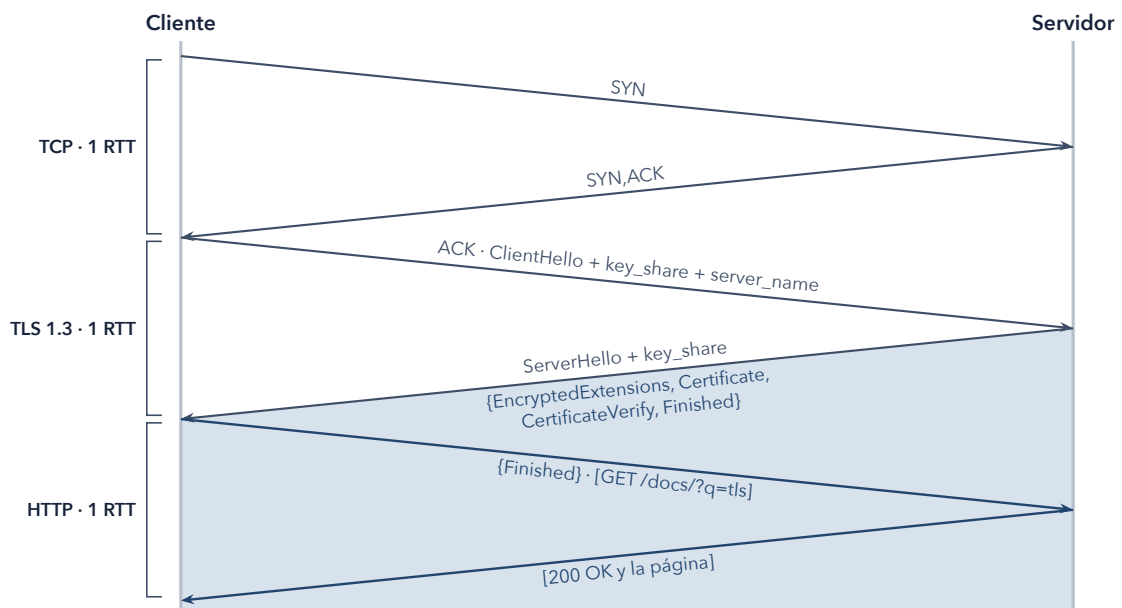
PARA REPASAR

1. ¿Qué parte de la URL viaja cifrada, y dónde viaja el host en claro?
2. ¿Por qué el DNS no entra en la cuenta de RTT?
3. ¿Por qué llegar a la IP y al puerto correctos no prueba la identidad del servidor?

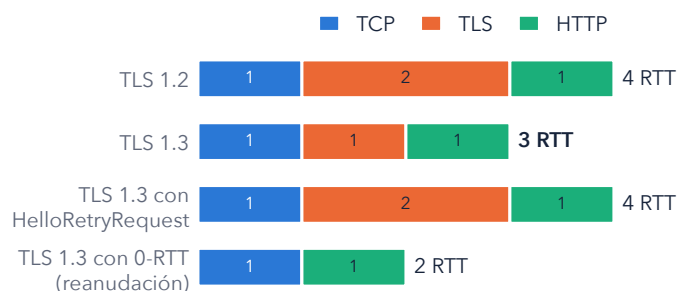
02 TLS 1.3 acuerda las claves en un solo viaje de ida y vuelta

Definición. El handshake es «una negociación inicial entre cliente y servidor que establece los parámetros de sus interacciones posteriores» RFC 8446, §1.1. En TLS 1.3 tiene tres fases RFC 8446, §2:

- **Intercambio de claves.** El ClientHello ofrece versiones y cifrados y lleva su parte del acuerdo de claves (key_share); el ServerHello responde con la suya. De ahí salen las claves, y lo que sigue va cifrado.
- **Parámetros del servidor.** EncryptedExtensions, y CertificateRequest si pide certificado al cliente, algo raro en la web MDN-TLS.
- **Autenticación.** Certificate lleva el certificado; CertificateVerify, una firma del handshake con la clave privada; Finished, un código de autenticación (MAC) del handshake que confirma las claves.



Ejemplo. Una visita nueva: tres RTT hasta la primera respuesta RFC 9293, fig. 6 · RFC 8446, fig. 1. Sobre celeste, lo que viaja cifrado: todo lo que sigue al ServerHello, con claves del handshake {...} o de la aplicación [...].



RTT hasta la primera respuesta, sin el DNS: cuenta propia sobre los diagramas de las RFC. En negrita, TLS 1.3 completo.

TLS 1.3 ahorra un RTT frente a TLS 1.2, que esperaba dos antes de los datos RFC 5246, fig. 1. Si el servidor no acepta el key_share del cliente, responde con HelloRetryRequest, el ClientHello se repite y el ahorro se pierde RFC 8446, §2.1.

Con una clave precompartida (PSK) de una conexión anterior, el cliente puede mandar la petición junto al ClientHello: es el modo 0-RTT RFC 8446, §2.2 y §2.3.

Confusión típica. Suponer que los datos 0-RTT tienen las garantías del resto. No tienen secreto hacia adelante, porque se cifran solo con claves de la PSK, y nada impide repetirlos entre conexiones RFC 8446, §2.3. Secreto hacia adelante: una clave de largo plazo filtrada después no compromete la sesión RFC 8446, §E.1.

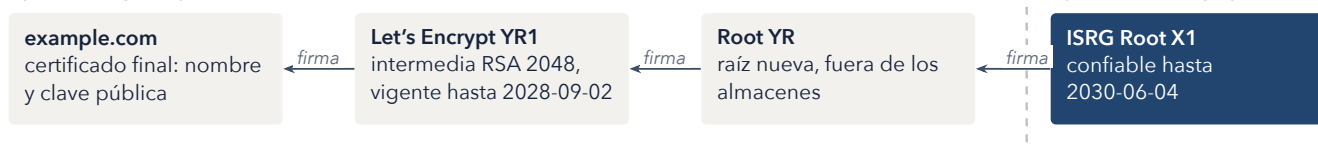
PARA REPASAR

1. ¿Qué lleva el ClientHello para que las claves salgan en un solo RTT?
2. ¿Qué prueba CertificateVerify y qué confirma Finished?
3. ¿Cuándo se pierde el RTT que ahorra TLS 1.3?
4. ¿Qué dos garantías les faltan a los datos 0-RTT?

03 El certificado ata un nombre a una clave, y la cadena lo avala

Definición. Validar una ruta de certificación es verificar el vínculo entre el nombre del sujeto y la clave pública del certificado final a partir de la clave de un ancla de confianza RFC 5280, §6.1. El certificado lleva además el emisor, la vigencia y los nombres DNS en `subjectAltName` RFC 5280, §4.1.2.5 y §4.2.1.6; solo uno con `ca` verdadero en `basicConstraints` firma otros RFC 5280, §4.2.1.9. El servidor manda su certificado y después los que lo certifican; el ancla puede faltar, porque el cliente ya la tiene RFC 8446, §4.4.2.

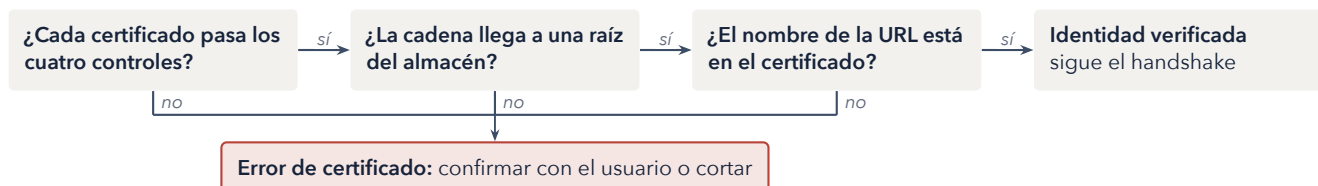
LO ENVÍA EL SERVIDOR



LO TIENE EL NAVEGADOR

Ejemplo. La cadena por defecto de un certificado RSA que emite Let's Encrypt YR1. Root YR aún no está en los almacenes de los navegadores: por eso la firma también ISRG Root X1, el ancla, en azul LE-cadenas.

De cada certificado, el cliente verifica la firma, la vigencia, que no esté revocado y que su emisor sea el sujeto del anterior RFC 5280, §6.1.3. Después compara el nombre de la URL con el certificado final.



Cualquier «no» lleva al error, con borde terracota: el navegador tiene que pedir confirmación al usuario o cortar la conexión RFC 9110, §4.3.4.

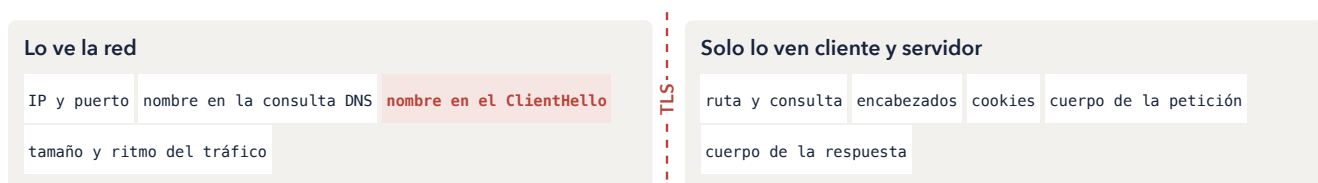
Confusión típica. Buscar el nombre del sitio en el Common Name (CN). El cliente arma la identidad de referencia como DNS-ID con el host de la URL; una CN-ID no debe usarse RFC 9110, §4.3.4.

PARA REPASAR

1. ¿Qué une un certificado X.509, y qué campo dice si puede firmar otros?
2. ¿Por qué cuatro controles pasa cada certificado de la ruta?
3. ¿Por qué Root YR necesita la firma de ISRG Root X1?
4. ¿Qué tiene que hacer el navegador si el nombre no coincide?

04 HTTPS protege el contenido y la identidad, no los metadatos

Handshake y certificado determinan juntos lo que HTTPS protege: el servidor queda autenticado y los datos, cifrados e íntegros RFC 8446, §1. Lo que viaja fuera de TLS lo ve cualquiera en la red.



A la izquierda, lo que ve la red aunque se use HTTPS RFC 9110, §4.3.3 · RFC 8446, §E.3; a la derecha, lo que queda dentro de TLS MDN-TLS. En rosa, el nombre en el ClientHello: va en claro porque solo se cifra lo que sigue al ServerHello RFC 8446, §1.2.

AMENAZA	QUÉ HACE HTTPS	FUENTE
Alguien en el medio lee o cambia el tráfico	Lo impide: cifrado, integridad y autenticación del servidor	MDN-TLS
Una respuesta DNS falsa lleva a otra IP	Lo detecta: el certificado no coincide con el nombre	RFC 9110, §4.3.4
Degradar a http la primera visita (SSL stripping)	HSTS lo evita en las siguientes; la lista de precarga, también en la primera	MDN-TLS
Un script pedido por http en una página https (contenido mixto)	El navegador pasa la petición a https o la bloquea	MDN-TLS
Repetir datos 0-RTT	TLS no lo impide; el servidor puede limitarlo	RFC 8446, §8
Deducir el contenido por el tamaño y el ritmo	No lo impide; el relleno de los registros lo dificulta	RFC 8446, §E.3

HTTPS impide o detecta los ataques al contenido y a la identidad; la primera visita, el 0-RTT y los metadatos necesitan otras defensas.

HSTS es el encabezado `Strict-Transport-Security`: pide al navegador que las próximas visitas vayan directo por https MDN-TLS. Con DoH, la consulta DNS viaja dentro de HTTPS y los equipos del camino no pueden interferir RFC 8484, §1.

PARA REPASAR

1. ¿Qué datos de una visita ve un observador de la red, y por qué?
2. ¿Por qué una respuesta DNS falsa no alcanza para suplantar un sitio https?
3. ¿Qué evita HSTS, y qué agrega la lista de precarga?

Referencias

- [RFC 8446] E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, IETF, 2018: <https://www.rfc-editor.org/rfc/rfc8446>.
- [RFC 5246] T. Dierks y E. Rescorla, *The TLS Protocol Version 1.2*, IETF, 2008: <https://www.rfc-editor.org/rfc/rfc5246>.
- [RFC 9110] R. Fielding, M. Nottingham y J. Reschke (eds.), *HTTP Semantics*, IETF, 2022: <https://www.rfc-editor.org/rfc/rfc9110>.
- [RFC 9293] W. Eddy (ed.), *Transmission Control Protocol (TCP)*, IETF, 2022: <https://www.rfc-editor.org/rfc/rfc9293>.
- [RFC 1035] P. Mockapetris, *Domain Names – Implementation and Specification*, IETF, 1987: <https://www.rfc-editor.org/rfc/rfc1035>.
- [RFC 8484] P. Hoffman y P. McManus, *DNS Queries over HTTPS (DoH)*, IETF, 2018: <https://www.rfc-editor.org/rfc/rfc8484>.
- [RFC 5280] D. Cooper y otros, *Internet X.509 Public Key Infrastructure Certificate and CRL Profile*, IETF, 2008: <https://www.rfc-editor.org/rfc/rfc5280>.
- [MDN-TLS] MDN Web Docs, *Transport Layer Security (TLS)*: https://developer.mozilla.org/en-US/docs/Web/Security/Defenses/Transport_Layer_Security.
- [LE-cadenas] Let's Encrypt, *Chains of Trust*, actualizada el 8 de julio de 2026: <https://letsencrypt.org/certificates/>.
- [LE-validación] Let's Encrypt, *How It Works*: <https://letsencrypt.org/how-it-works/>.